Knowledge Transfer — structured-pruning

For anyone picking up this codebase — including future-you six months from now. Read this before changing `prunelib/`.

1. What this is and why it exists — and what it *isn't*, since this changed

Scope note, read this first if you haven't looked at this repo in a while: this package started as a full replacement for the buggy CNN pruning scripts in `pruning_framwork_v4`. Those scripts were subsequently fixed *directly, in that repo*, with more complete method coverage (K-means, SVD, hybrid sequencing) than this package's `prunelib` has. **`pruning_framwork_v4` is now canonical for CNN pruning.** This package's job is narrower: Transformer pruning (the original extension experiments, plus `prunelib.transformer`, merged in 2026-11-12 from the standalone `transformer_pruning` repo), and a tested reference implementation of the mask-then-compress design (section 4 covers both). See the top of `README.md` for the current framing — if a change you're considering is "add K-means to `prunelib`," it belongs in `pruning_framwork_v4` instead, not here.

The history below is why the code looks the way it does; it's not a claim about what this repo currently supersedes.

This package is a reference implementation of the structured-pruning methods from six of my papers (see `PUBLICATIONS.md`), originally built because four earlier repos (`pruning_framwork`, `_v2`, `_v3`, `_v4`) contained real, confirmed bugs — including one in the exact method the papers are named after.

That history matters for how this code is written, not just as trivia. Every non-obvious design choice in `prunelib/` exists because of a specific, verified bug in the old code. If a change looks like it's undoing one of these choices, it's probably reintroducing the bug. Section 3 maps each choice to the defect it prevents.

The short version of the history: `pruning_framwork_v4/facilitate_pruning.py` computed a Max-3 saliency score into a tensor called `channel_norm`, then selected which channels to prune by comparing a *different* tensor, `channel_norm_temp`, which was never updated and still held the plain L1 norm. The score was computed, stored, returned — and silently ignored. Six more defects (D2–D7) were found in the same file and one neighboring one. None of them were exotic — an off-by-one, a top-k tracker that only updates on new maxima, a boolean tensor passed to `torch.norm()`. All are listed in detail in section 3. All are pinned by a test in `tests/`.

The real irony: the *papers* are fine. Max-3 and L1 produce measurably different accuracy curves in the published results, which they couldn't if v4's broken code had generated them. Working Max-3 code existed — it just was never pushed to GitHub. It isn't recoverable; this package's `prunelib` was the reference implementation until `pruning_framwork_v4` got fixed in place — see the scope note above for where things stand now.

2. Repository layout

```
prunelib/
    saliency.py   Max-k / L1 / L2 / CSD / random scoring behind one registry
                  (register_saliency), Conv2d or Linear weights; prune_count
    surgery.py    Conv/BN/FFN/attention-head structural surgery, built on three
                  slicing helpers (_slice_conv, _slice_bn, _slice_linear)
    masking.py    two-phase mask-then-compress workflow (torch.nn.utils.prune)
    vgg.py        VGG wiring: build_vgg16, prune_vgg_layer, mask_vgg_layer,
compress_masked_vgg
    scanners.py   distance-metric registry (register_metric) + co-activation scanning
    evaluate.py   parameter counts + measured latency
    transformer/  model-level Transformer pruning, merged from transformer_pruning
                  (section 4). Layered, lowest first -- each may import only
                  the ones above it, and any of them may import core prunelib:
        models/       adapters (BERT/RoBERTa layout, resolve_adapter), discover_layers
        measurement/  hooks, FFNCalibrator, HeadCalibrator, ActivationRecorder
        analysis/     scorers, NetworkSaliencyScanner, OV-circuit math,
                      JaccardTwinFinder, kmeans_assign
        selection/    FFNContext/HeadContext/Selection, budget rule, FFN + head selectors
        surgery/      rebuild_linear, FFNSurgeon, AttentionSurgeon
        pipeline/     allocation strategies, prune_* entry points
experiments/
    00_demo.py                    full pipeline, seconds, no dependencies beyond torch
    01_vgg_cifar10_sweep.py       VGG16/CIFAR-10 -- see section 4, this one has three run
modes
    02_bert_sst2_sweep.py         BERT FFN pruning
    03_head_redundancy.py         attention head distance scan
    04_coactivation.py            synthetic co-activation demo
    05_ordering.py                does the CNN ordering result transfer?
    06_transformer_techniques.py  prunelib.transformer end to end: calibrated global
                                  head + FFN pruning with bias compensation, twin merge
archive/
    legacy_pipeline/        corrected rebuild of pruning_framwork_v4's original
        config.py, data.py, driver scripts. Superseded by direct fixes to that
        model.py, train.py, repo (which has more complete method coverage) --
        pipeline.py         kept here as a tested mask-then-compress example,
                             not a live alternative. Moved under archive/
                             2026-09-21. See LEGACY_PIPELINE_MIGRATION.md.
tests/              59 core tests, one per historical defect or behavior
    transformer/    199 tests for prunelib.transformer (BERT-shaped stub model in
                    conftest.py -- no HuggingFace needed)
```

Core `prunelib` is ~1,000 lines; `prunelib.transformer` ~3,200 (heavily docstringed). Small modules on purpose — each does one thing.

3. Design decisions, and the defect each one prevents

If you're refactoring `prunelib`, read this table first.

Decision	Defect it prevents	Where
`compute_score()` is the *only* way to get a saliency score; `select_prune_indices()` takes a score tensor, never a weight tensor	D1: score computed one place, selection reads a different, stale tensor	`saliency.py`
`torch.topk` instead of a hand-rolled "track top-3" loop	D2: manual tracker only rotated on a *new* max, silently dropping mid-range values	`saliency.py::max_k_saliency`
Per-kernel top-k summed across input channels in one vectorized op (`.sum(dim=2)` then `.sum(dim=1)`), not an accumulator reset "once per output channel" but mutated inside an inner loop	D3: score accumulated from monotonically-growing partial maxima across input channels — not the intended quantity at all	`saliency.py::max_k_saliency`
Kernel height and width read from their own tensor dimensions (`kh, kw = weight.shape[2], weight.shape[3]`), never reusing one dimension for both	D4: original iterated `range(size[2])` for *both* height and width, dropping columns on non-square kernels	`saliency.py::max_k_saliency`
`prune_ffn_block` and `prune_conv_bn` raise `ValueError` immediately if shapes don't match, before touching any tensor	D5: a shape bug surfaced as a confusing `IndexError` deep inside an unrelated loop instead of at the actual mistake	`surgery.py`, both functions' first lines
Surgery is pure advanced indexing (`weight.index_select(0, keep_idx)`); no manual per-channel copy loop with a hand-maintained destination counter	D6: the old copy loop's destination index never incremented, so every surviving channel wrote to index 0	`surgery.py::prune_conv_bn`
Same as D6 — no manual loop, so there's no "off-by-one before first use" to have	D7: `fin_new = fin_org` then `fin_new += 1` before the variable was ever read	`surgery.py` (structurally can't recur)
Pruning a VGG layer replaces modules in-place inside `model.features` rather than rebuilding a whole model from a `feature_list` and copying weights in	This *is* D6/D7's root cause, one level up — the old code's entire "reconstruct then copy" pattern is gone, not just patched	`experiments/01_vgg_cifar10_sweep.py::prune_vgg_layer`, `vgg.py::compress_masked_vgg`
Masking uses `torch.nn.utils.prune.custom_from_mask` — one call, full mask computed up front — instead of subclassing `BasePruningMethod` with a `compute_mask` that reconstructs the mask from global state on every call	The systemic bug across *every* driver script in old `pruning_framwork_v4`: a `layer_number` global was supposed to tell `compute_mask` which layer it was on, and the line updating it was commented out in all six scripts, so it silently stayed 0 forever (see `GITHUB_AUDIT.md` section 11)	`masking.py::mask_channels`
`mask_vgg_layer` scores and selects only from `surviving_channels` (not-yet-masked channels), never all channels	Without this, an already-masked channel's weight is zero — the lowest possible score under every criterion — so it wins re-selection on every later iteration, and the pruning schedule advances far slower than the requested fraction implies	`vgg.py::mask_vgg_layer`
Saliency methods and distance metrics live in registries (`register_saliency`, `register_metric`); re-registering a name raises unless `overwrite=True`	Adding a criterion used to mean editing library internals, and a typo could silently shadow a built-in	`saliency.py`, `scanners.py`

Decision	Defect it prevents	Where
One `prune_count(n, fraction)` and one `keep_indices(scores, n)` used by every schedule; no caller hand-rolls `{"k": 3} if method == "max_k"` or a complement list	Six copies of the same rounding/complement logic had started to drift (some `int()`, some `round()`)	`saliency.py`, all of `vgg.py` and `experiments/`
Surgery builds replacements on the source's device and dtype, and rejects grouped/depthwise convs	Surgery on a GPU/fp16 model quietly returned CPU float32 layers; a grouped conv was silently rebuilt as a dense one	`surgery.py::_slice_conv`
`prunelib.transformer` builds every resized Linear through `rebuild_linear` (dtype, device, `requires_grad` preserved)	A bare `nn.Linear` converted an fp16 FFN to fp32 and un-froze frozen layers during a "heal"	`transformer/surgery/linear_ops.py`
Transformer budgets `round`, never truncate (`keep_count`)	`int()` always pruned one unit more than asked — a one-directional bias compounding over a whole-model sweep	`transformer/selection/budget.py`
Every calibration/recording hook drops padding through one `flatten_tokens`	Counting `[PAD]` positions dragged means toward the pad output and made every neuron look co-active with every other	`transformer/measurement/hooks.py`
Twin detection uses true Jaccard (`intersection / union`), and twin chains resolve by union-find onto the lowest index	The original notebook divided by `fires_A + fires_B`; pair-order-dependent resolution merged a neuron *into an already-removed neuron*	`transformer/analysis/redundancy.py`, `transformer/selection/ffn_selectors.py::TwinRedundancySelector`

If you ever find yourself writing a loop that tracks a running max/min by hand, or a loop with a manually-incremented destination index into a tensor — stop. Both patterns have already caused a real bug in this project's history. Use `torch.topk` / `torch.sort` and advanced indexing instead.

4. Module walkthrough

`prunelib/saliency.py`

Five scoring functions (`max_k_saliency`, `l1_saliency`, `l2_saliency`, `csd_saliency`, `random_saliency`), all with signature `(weight: [out,in,kh,kw] or [out,in]) -> [out]`, routed through `compute_score(weight, method=...)` via a registry (`register_saliency`, `available_methods()`). Each scorer's own defaults apply, so `compute_score(w, "max_k")` already means k=3 — callers never pass method-specific kwargs to get the paper's setup. `prune_count(n, fraction)` is the one rounding rule ("round, at least 1") every iterative schedule uses. `csd_saliency` (L1 dispersion from the row mean, from the "Enhancing Deep Compression of CNNs" paper) is shared with `prunelib.transformer.CSDScorer` — same function, both sides. `select_prune_indices` and `keep_indices` operate on the *score* tensor, not the weight — this separation is what makes D1 structurally impossible to reintroduce, since there's no code path where a caller has both a weight tensor and a stale score tensor in scope at the same time.

`max_k_saliency` is the one to understand well: it flattens each kernel to 1D, takes the top-`k` magnitudes with `torch.topk`, sums them (`kernel_scores`, shape `[out, in]`), then sums across the input-channel axis exactly once (`channel_scores`, shape `[out]`). That's the whole algorithm. If a change here doesn't fit in three lines of einsum-shaped reasoning, it's probably reintroducing a shape bug — this function has been the site of 4 of the 7 historical defects.

`prunelib/surgery.py`

Three functions: `prune_conv_bn` (Conv2d + optional BatchNorm2d + optional next Conv2d), `prune_ffn_block` (Linear → activation → Linear), and `prune_attention_heads` (Q/K/V rows + output-projection columns, heads to **keep**). All three are thin compositions of `_slice_conv` / `_slice_bn` / `_slice_linear`, which keep chosen rows and/or columns and build the new module on the source's device and dtype. All build *new* modules of the correct smaller size and copy the kept rows/columns in — they never mutate the originals. Both validate the seam (that the tensor shapes on either side of the cut actually match) before doing anything, and raise `ValueError` with a specific message if not.

One deliberate design choice worth knowing: `prune_conv_bn` copies BatchNorm's running mean/var across to the new, smaller BN layer. The original code didn't do this at all — pruned models ran without error but produced silently wrong activations until the next full training pass reset the running stats. This isn't "more correct" in some abstract sense, it's specifically there because that failure mode doesn't throw, so nothing would tell you it happened.

`prunelib/masking.py`

The two-phase mask-then-compress workflow: `mask_channels` zeroes a channel's weight *and* bias via `torch.nn.utils.prune.custom_from_mask` (non-destructive — shapes don't change, and it composes correctly across repeated calls on the same layer, which is what makes it safe to call once per pruning iteration). `commit_mask` bakes the zeros in permanently. `compress_masked_conv_bn` reads back which channels ended up all-zero and calls `surgery.prune_conv_bn` on the survivors — same tested surgery code as everywhere else, just fed indices derived from a committed mask instead of a fresh score.

The point of this two-phase split: you can fine-tune or evaluate with the mask active, across as many iterations as you want, before committing to an architecture change. Gradients naturally don't reach masked positions (`weight = weight_orig * mask`, so `d(loss)/d(weight_orig) = d(loss)/d(weight) * mask`, which is zero wherever the mask is zero) — no extra bookkeeping needed to keep pruned channels from "coming back" during fine-tuning.

`tests/test_masking.py::test_compress_masked_conv_bn_matches_direct_surgery` is the test to read if you're unsure this is equivalent to just calling `prune_conv_bn` directly — it proves the two paths produce numerically identical output.

`prunelib/vgg.py`

VGG-specific wiring on top of the layer-level primitives above. `build_vgg16` wraps `torchvision.models.vgg16` (using `weights=`, not the deprecated `pretrained=True/False`), and is the only function here that imports torchvision — lazily — so the rest works on any model whose `.features` is a Conv/BN/ReLU `Sequential` without the optional dependency. Seam lookup and write-back are shared helpers (`_require_prunable`, `_shrink_seam`). `prune_vgg_layer` is the direct, one-shot version

(score → select → surgery, immediately). `mask_vgg_layer` / `compress_masked_vgg` are the two-phase version — see the design-decision table in section 3 for the correctness fix `mask_vgg_layer` needed (excluding already-masked channels from re-selection).

Both `prune_vgg_layer` and `mask_vgg_layer` refuse to touch the *last* conv layer (it feeds `classifier[0]`, a `Linear`, not another `Conv2d` — see section 6).

`archive/legacy_pipeline/`

Moved under `archive/` 2026-09-21 (was `legacy_pipeline/` at the repo root; import as `archive.legacy_pipeline`) to make explicit that this package is `prunelib` plus the Transformer experiments first, not a pipeline you run in place — see README.md's opening line. A corrected rebuild of `pruning_framwork_v4`'s original six driver scripts, done in parallel with this package, before those scripts got fixed directly in that repo. **Now superseded**: `pruning_framwork_v4`'s own fix has more complete method coverage (K-means, SVD, hybrid sequencing) than `legacy_pipeline` does, since `legacy_pipeline` is built on this package's `prunelib` (Max-k/L1/L2/random only). Don't extend `legacy_pipeline` to add parity with `pruning_framwork_v4` — that work belongs in `pruning_framwork_v4` itself. What's still worth reading here: `pipeline.py::run_pruning` is a complete, tested example of the mask-then-compress workflow applied to a whole VGG16 end to end (mask every prunable layer per iteration → fine-tune → evaluate → repeat → compress once at the end), and `tests/test_legacy_pipeline.py` is the regression test for that. See `LEGACY_PIPELINE_MIGRATION.md` for the full defect-by-defect mapping from the original scripts. `run_pruning` accepts injected `model=`, `loaders=` and `device=` (defaulting to what the config describes), and datasets are added with `data.register_dataset`.

`prunelib/scanners.py`

`pairwise_distance_matrix(vectors, metric)` implements Manhattan/Euclidean/ Cosine — the three metrics compared in Paper 2 — through a registry (`register_metric`).
`prunelib.transformer.AttentionHeadAnalyzer` uses this same registry for head distances. For *CNN channels* it computes distances only: K-Means channel selection for CNNs stays in `pruning_framwork_v4` (see section 6). The FFN-neuron version of that paper's K-Means rule does exist here, as `prunelib.transformer.WeightClusterRedundancySelector` (built on `kmeans_assign`).

`CoActivationScanner` computes pairwise Jaccard similarity on boolean firing masks, with a `firing_rate_ceiling` (default 0.9) that excludes near- universal-firing units before computing similarity. This exists because a unit that fires on 95%+ of tokens will show spuriously high overlap with *any* other frequently-firing unit — that's an artifact of both units being non-selective, not evidence of redundancy. This hasn't been validated against real model activations yet (see section 6) — the logic is tested against synthetic masks in `tests/test_scanners.py` and demonstrated in `experiments/04_coactivation.py`, but nobody has confirmed the 0.9 default is the right threshold on an actual fine-tuned model.

`prunelib/transformer/`

Merged 2026-11-12 from the standalone `transformer_pruning` repo (its own `docs/KNOWLEDGE_TRANSFER.md` is the long-form reference for this subpackage; this section is the map). Where core `prunelib` is stateless tensor-in/tensor-out, this works on a live model. Import everything from `prunelib.transformer`; the subpackages are internal layout.

The pipeline is **measure** → **select** → **cut**, with each step behind an interface:

- **Reaching into a model — `models/`.** `FFNLayerAdapter` (get/set an FFN pair) and `AttentionLayerAdapter` (get/set the Q/K/V/O quartet, per-layer head count) are split because no consumer needs both. `BertLayerAdapter` covers BERT and RoBERTa. Nothing names it directly: every consumer calls `resolve_adapter(model)`, which consults a registry (`register_adapter`) and unwraps HF task models via `base_model`, so a `BertForSequenceClassification` can be passed anywhere.
- **Measuring — `measurement/`.** `FFNCalibrator` streams per-neuron activation moments (`CalibrationStats`: mean, mean_abs, rms), and `collect_many` does every layer in one pass. `HeadCalibrator` gives per-head contribution moments (`HeadStats`) plus Michel et al.'s gradient importance `E|dL/dξ_h|`, placing the head gate with a pre-hook (transformers 5.x dropped `head_mask`). `ActivationRecorder` keeps full per-token firing patterns, which twin detection needs. All share `run_hooked`, which always removes its hooks and restores train/eval mode.
- **Deciding — `selection/`.** Selectors get a context, not a bare weight: `FFNContext` carries `W_in`, `W_out`, biases and optional stats, because a neuron's contribution is `W_out[:, i] · act(W_in[i]·x + b_i)`. FFN criteria: `SaliencySelector(scorer)`, `InOutNormSelector`, `ActivationAwareSelector`, `TwinRedundancySelector`, `WeightClusterRedundancySelector`. Head criteria: `RedundantHeadSelector`, `OVNormHeadSelector`, `ActivationAwareHeadSelector`, `GradientHeadSelector`. FFN selectors return a `Selection` (keep list plus optional `merge_map`).
- **Cutting — `surgery/`.** `FFNSurgeon` slices, folds merges and applies bias compensation. `AttentionSurgeon` removes whole heads and can also compensate. It can't merge, because two heads that attend alike can still read and write different subspaces.
- **Orchestrating — `pipeline/`.** `prune_ffn_layer` / `prune_attention_layer` act on one layer; `prune_model_ffn` / `prune_model_attention` act on the whole stack under an `AllocationStrategy`: `"uniform"`, `"global"` (rank every unit model-wide after per-layer normalization, with a `min_keep_ratio` floor), or your own.

Two operational rules to remember. **Cut heads before FFNs, and re-calibrate in between**, because the head cut changes every downstream FFN's input distribution. **Calibration stats go stale after surgery**: the contexts and surgeons check neuron/head counts and raise rather than reuse them silently (use `stats.select(kept)` if you mean to).

Seams with core (enforced by `tests/transformer/test_package_layout.py`: core never imports `prunelib.transformer`): - `CSDScorer` calls `prunelib.csd_saliency`. - `RegisteredScorer("l1")` turns any core-registered method into a `SaliencyScorer`. - `AttentionHeadAnalyzer.compute_distance_matrix` uses `prunelib.pairwise_distance_matrix`. - pandas is optional, imported only inside `NetworkSaliencyScanner.scan()` (the `[analysis]` extra).

`prunelib/evaluate.py`

`count_params`, `count_encoder_params` (params under a named submodule, e.g. `.encoder`, excluding embeddings), and `measure_latency` (wall-clock, warmup + averaged iterations, CPU by default). Nothing subtle here, but note: `measure_latency`'s numbers are hardware- and batch-size-dependent. Don't hardcode a specific multiplier anywhere that isn't clearly labeled with the machine/conditions it came from — see the README's note about the 1.8x vs 2.13x latency discrepancy for why this matters in practice.

`experiments/`

Three-tier pattern used across the VGG and BERT experiments:

- **`--smoke`** — fully synthetic data and a tiny hand-built model (no `torchvision` / `transformers` model classes involved for `01`). Runs in under a second. This is what CI runs on every push.
- **`--tiny-check`** (currently only on `01`) — the *real* model class (`torchvision.models.vgg16`) with randomly-initialized weights and `FakeData` instead of a real dataset. No network access needed, but it exercises the actual code path `run_full()` uses. Takes minutes, not seconds — not run in CI, but should be run manually after any change to `prune_vgg_layer` or `build_vgg16`.
- **no flag / `run_full()`** — the real thing. Downloads real weights and a real dataset. Needs internet access and has not been executed anywhere yet (see section 6) — everything up to this point has only been verified mechanically.

If you add another experiment that touches a real pretrained model, follow this same three-tier pattern rather than inventing a new one.

`06_transformer_techniques.py --smoke` runs the `prunelib.transformer` pipeline on a random-init `BertForSequenceClassification`: global gradient-ranked head pruning, re-calibration, global activation-aware FFN pruning (both with bias compensation), then a twin merge. Random weights have no real twins, so it plants one exact duplicate neuron to exercise the merge path. It has no real-run mode yet. The MRPC accuracy harness (`experiments/accuracy/` in `transformer_pruning`) was not ported.

5. Testing conventions

Every test in `tests/` is named after either a historical defect (`test_d1_...`, `test_d6_...`) or a specific behavior guarantee (`test_ffn_block_shrinks_and_preserves_values`). When you fix a bug in this codebase, the workflow is:

1. Write a test that fails against the buggy code, named `test_dN_<what_broke>` if it's a regression of a known historical pattern, or descriptively otherwise.
2. Fix the bug.
3. Confirm the test passes and nothing else broke: `pytest tests/ -v`.

This means the test suite doubles as a defect log — reading `tests/` top to bottom tells you most of this codebase's incident history without needing `GITHUB_AUDIT.md` open at the same time. Keep it that way: don't delete a defect-named test even if you refactor the code it guards, unless you're certain the refactor makes the bug class structurally impossible (as, e.g., switching to `torch.topk` made D2 impossible to reintroduce even accidentally).

`tests/transformer/` follows the conventions of the repo it came from instead: tests are named for the behavior they guarantee (e.g. exact mean preservation under bias compensation, merge of identical neurons being output-preserving, gradient importance matching finite differences). They run against `conftest.py::StubBert`, a hand-built module tree with BERT's `encoder.layer[i]` shape, so they need no HuggingFace and run in under a second. `test_package_layout.py` treats the folder layering as architecture and fails on an upward import.

Run everything: `PYTHONPATH=. pytest tests/ -v` (258 tests, 1 skipped without CUDA; the full suite including the `archive/legacy_pipeline` end-to-end tests takes a few minutes since those actually train a tiny VGG16 — the pure-`prunelib` tests alone are still ~2s).

6. Known gaps — read this before claiming something works

Being explicit about what's *not* done is as important as documenting what is, given this project's history of a paper's headline result having no correct public implementation. Current state, honestly:

- **K-Means, SVD, and hybrid-sequencing criteria are not in this package's `prunelib`, and that's now by design, not a gap to fill here.** They exist, fixed and tested, in `pruning_framwork_v4`. `scanners.py` has the distance metrics `pairwise_distance_matrix` needs, but adding a `k_means_select()` here would create a second, competing implementation of something `pruning_framwork_v4` already does more completely — if you want K-means/SVD/hybrid pruning, use that repo, don't rebuild it here.
- **`prune_vgg_layer` can't prune the last conv layer** — it feeds `classifier[0]` rather than another `Conv2d`, and resizing that Linear layer's `in_features` isn't wired up. Straightforward to add; just not done.
- **`run_full()` in `experiments/01` has never been executed.** It's real code, verified mechanically via `--tiny-check` (real model class, no network), but nobody has run it against actual CIFAR-10 and actual ImageNet-pretrained weights yet. Don't cite a specific accuracy-drop number from this codebase until that's been done.
- **`experiments/02` and `03` are the same situation** — real `transformers` model classes, verified in `--smoke`, never run against a real fine-tuned checkpoint or real SST-2 data.
- ~~**FFN saliency scoring is a reshape hack, not a first-class API.**~~ **Fixed 2026-09-21.** `saliency.py`'s scorers now accept a 2D Linear weight `[out_features, in_features]` natively (via `_channel_view`), alongside the original 4D Conv2d shape. `experiments/02_bert_sst2_sweep.py::_score_ffn_neurons` no longer reshapes around it.
- ~~**No attention-head *pruning* surgery exists**~~ **Fixed 2026-09-21.** `surgery.py` now has `prune_attention_heads`, the equivalent of `prune_conv_bn` for a multi-head attention block (touches Q/K/V rows and the output projection's columns simultaneously). Detection → pruning is now wired too, at the model level: `prunelib.transformer.RedundantHeadSelector` (OV-circuit or query-weight cosine) with `prune_attention_layer`. `experiments/03` itself still only *reports* distances.
- **`CoActivationScanner`'s 0.9 firing-rate ceiling is a design choice, not a validated threshold.** Only tested against hand-constructed synthetic masks so far. (`prunelib.transformer.JaccardTwinFinder` is the newer, real-activation counterpart: it reports dead neurons separately and records real HF activations through `ActivationRecorder`. The two overlap and haven't been consolidated.)
- **No `prunelib.transformer` criterion has a reproduced accuracy number in this repo.** The mechanics are tested (and the source repo reports a check against a real transformers `BertForSequenceClassification` where pruned logits matched zero-gating the same heads to 1e-8 -- not re-run here), but the MRPC accuracy scripts were not ported and nothing here has been fine-tuned → pruned → healed end to end. Don't cite a criterion comparison from this repo until that's done.

- **Two definitions of "Max-k on a Linear layer" coexist.** Core `"max_k"` treats a Linear weight as a 1x1 conv (top-k over each input's one-tap kernel), so on an FFN it equals L1. `prunelib.transformer.TopKMagnitudeScorer(k)` sums each neuron's k strongest incoming connections. They are deliberately not reconciled until the papers' intended definition is confirmed. Pick one explicitly in any experiment, and note that `RegisteredScorer("max_k")` gives the core (L1-equivalent) behavior.
- **Two `prune_attention_heads` functions.** `prunelib.prune_attention_heads(q, k, v, o, keep_heads, num_heads)` is tensor-level and takes heads to **keep**. `prunelib.transformer.prune_attention_heads(model, layer, head_indices)` is model-level, takes heads to **drop**, and updates the layer's head-count bookkeeping. Check which one you imported.

7. Extending this codebase — where new work goes

- **New scoring method** (e.g. a second-order/Hessian-based saliency): write a function matching the existing signature (`weight: [out,in,kh,kw] -> Tensor[out]`) and register it with `@prunelib.register_saliency("name")` -- no edit to `saliency.py` needed (distance metrics work the same way via `register_metric`). Add a test in `tests/test_saliency.py` following the existing pattern (independent brute-force comparison where feasible, like `test_d2_matches_bruteforce_topk`).
- **New surgery type** (`prune_attention_heads` is the worked example now): add to `surgery.py` , validate the seam and raise before mutating anything (follow `prune_ffn_block` 's pattern exactly), add a test that checks *values* survive correctly post-surgery, not just shapes (see `test_d6_conv_values_are_correct_not_just_shape` for why shape-only tests aren't enough — that's literally how D6 shipped originally), and where feasible verify against a real HF model forward pass, not just plain `nn.Linear` (see `test_prune_attention_heads_against_a_real_hf_bert_model`).
- **New Transformer criterion**: subclass `ImportanceSelector` (FFN) or `HeadImportanceSelector` (heads) and implement `importance(ctx)` . That alone makes it work with uniform *and* global allocation. Need more than a per-unit score (merges, structural rules)? Subclass `NeuronSelector` / `HeadSelector` and override `select` , and accept that global allocation will reject it. Pure weight scores can also just be core methods (`register_saliency`) wrapped in `RegisteredScorer` . Tests go in `tests/transformer/` , against `StubBert` .
- **New model family** (GPT-2, T5, ViT, ...): write an adapter implementing `FFNLayerAdapter` and/or `AttentionLayerAdapter` , and call `register_adapter(matches, factory)` once. No consumer changes.
- **New allocation policy**: subclass `AllocationStrategy` and pass it as `allocation=` . It sees neurons and heads through the same `PrunableStructure` interface, so one class serves both.
- **New experiment**: follow the three-tier `--smoke` / `--tiny-check` / full pattern from section 4. Add the `--smoke` invocation to `.github/workflows/tests.yml` so CI actually exercises it.
- **A new CNN pruning criterion (K-means, SVD, second-order, anything matching the papers) belongs in `pruning_framwork_v4` , not here.** That repo is canonical for CNN methods and already has more coverage than this package's `prunelib` . Adding one here would create a second, divergent implementation of the same idea — see the scope note at the top of section 1.

- **Transformer-extension work** (wiring `experiments/03`'s detected redundant-head pairs into actual `prune_attention_heads` calls, deciding a real criterion rather than raw query-weight distance, validating `CoActivationScanner`'s threshold against real activations, reproducing accuracy numbers for the `prunelib.transformer` criteria) is this repo's actual remaining job — see the gaps list in section 6 for what's still open there. `transformer_pruning` has been merged in as `prunelib.transformer`, so do new Transformer work here. Changes to the standalone repo's copy would diverge from this one.

8. Onboarding checklist

First: check the scope note at the top of section 1. If you came here to add or fix a CNN pruning criterion, you probably want `pruning_framwork_v4` instead.

```
git clone https://github.com/thakerpragnesh/structured-pruning.git
cd structured-pruning
pip install -e ".[dev,vision-experiments,transformer-experiments]"
pytest tests/ -v                        # 258 tests (core + tests/transformer/)
python experiments/00_demo.py           # full pipeline, seconds
python experiments/01_vgg_cifar10_sweep.py --smoke
python experiments/06_transformer_techniques.py --smoke   # prunelib.transformer end to end
```

Read section 3 of this document before touching `prunelib/saliency.py`, `prunelib/surgery.py`, or `prunelib/masking.py`, and the `prunelib/transformer/` part of section 4 before touching anything under that folder. Read section 6 before writing anything that implies full VGG16/CIFAR-10 results or BERT fine-tuning results are already produced by this codebase — the mask-then- compress *mechanism* is tested and correct; specific accuracy numbers from real training runs haven't been produced here yet.

9. Glossary

- **Saliency score** — a per-channel (or per-neuron) number estimating how much that channel contributes to the model's output; lower means "safer to prune."
- **Structural surgery** — physically resizing a layer (fewer real rows/ columns in a real tensor) as opposed to zero-masking, which keeps the tensor the same size and just sets some entries to zero. Surgery makes the model smaller and faster; masking only makes it sparser.
- **Seam** — the shape contract between two connected layers (a conv's output channels matching the next conv's input channels, or an FFN's `fc1.out_features` matching `fc2.in_features`). "Seam mismatch" is this codebase's term for what happens when surgery on one layer isn't propagated to the layer downstream of it.
- **FFN block** — the two-Linear-layer (expand, then project back down) feed-forward sub-layer inside each Transformer block.
- **Co-activation** — two units (neurons, heads) firing on the same inputs; a redundancy signal independent of whether their weights look similar.
- **Firing rate** — the fraction of tokens/inputs a given unit is "active" on, used to exclude near-universal-firing units from co-activation analysis (see section 4).

- **Calibration** — running a small set of real inputs through the model with hooks attached, to measure how units actually behave (activation moments, head contributions, gradient importance), as opposed to judging them from weights alone.
- **Bias compensation** — when a unit is removed, adding its *mean* output contribution (`W_out[:, i]·E[a_i]`, or `W_O[:, h]·E[ctx_h]` for a head) to the next layer's bias. Turns zero-ablation into mean-ablation; the layer's mean output is preserved exactly on the calibration distribution.
- **Merge** — folding a removed FFN neuron's output column into a surviving "twin" (scaled), so its signal is carried rather than deleted.
- **OV circuit** — a head's `W_O[:, h] · W_V[h]`: the map from what it attends to onto what it writes into the residual stream. Its norm bounds how much the head can matter.
- **Uniform vs. global allocation** — uniform cuts every layer by the same fraction; global ranks every unit in the model together (after per-layer score normalization), so redundant layers give up more.
- **Adapter** — the object that knows one model family's layout (`encoder.layer[i].intermediate.dense`, ...), so nothing else in `prunelib.transformer` has to.
- **Mask-then-compress** — the two-phase pruning design in `masking.py`: zero a channel's contribution via reparametrization first (reversible, same tensor shapes, safe to fine-tune against), and only physically resize the model once, at the end, via `compress_masked_conv_bn`. As opposed to one-shot surgery (`prune_conv_bn` called directly), which resizes immediately on every pruning decision.